Binary Search & Binary Search on the Answer

Halve the search space — on arrays, on numbers, and on answers

Learning goals

- Implement binary search **without off-by-one bugs** using an invariant
- Use `lower_bound` / `upper_bound` for counting and nearest-value queries
- Recognize **monotone predicates** and binary search on the answer
- Solve "minimize the maximum" / "maximize the minimum" problems
- Binary search over real numbers
- Analyze: O(log(range)) iterations × cost of the check

Motivation: CSES 1620 — Factory Machines

n machines; machine i makes one product every k_i seconds. Minimum time to make t products? n ≤ 2·10^5, t, k_i ≤ 10^9.

- Simulating second by second: up to 10^18 seconds ×
- But: "can we make t products in time T?" is easy: Σ ⌊T / k_i⌋ ≥ t
- And if time T works, any T' > T works → **monotone**
- Binary search on T: ~60 checks × O(n) ✓

The core idea

Searching sorted data: compare with the middle, throw away half.

```
a = 1 3 4 7 9 12 15 20      find 12
    [               ]       mid = 7  -> go right
              [     ]       mid = 12 -> found
```

Each step halves the range: n → n/2 → ... → 1 in $\lceil \log_2 n \rceil$ steps.

n = 10^9 → 30 steps. n = 10^18 → 60 steps.

The universal formulation

Every binary search finds the **boundary** of a monotone predicate:

```
index:   0 1 2 3 4 5 6 7
P(i):    F F F F T T T T
                 ^ first true
```

Find the first i with P(i) true, where P is false…false, true…true.

`lower_bound(x)` is this with P(i) = (a[i] ≥ x).

Template: invariant style

Invariant: P(lo) is **false**, P(hi) is **true** (use sentinels outside the range).

```
// first index in [0, n) with P true, or n if none
int lo = -1, hi = n;                // P(-1)=false, P(n)=true (virtual)
while (hi - lo > 1) {
    int mid = lo + (hi - lo) / 2;
    if (P(mid)) hi = mid;
    else        lo = mid;
}
// answer: hi
```

- Loop ends with hi = lo + 1: the boundary
- No `mid ± 1`, no `<=` debates, terminates always

Trace: lower_bound of 8

```
a:   1 3 4 7 9 12 15 20      P(i) = a[i] >= 8
i:   0 1 2 3 4  5  6  7
lo=-1 hi=8  mid=3  a=7  F -> lo=3
lo=3  hi=8  mid=5  a=12 T -> hi=5
lo=3  hi=5  mid=4  a=9  T -> hi=4
lo=3  hi=4  stop    answer hi = 4 (a[4] = 9)
```

STL binary search on sorted arrays

```
int i = lower_bound(all(a), x) - a.begin();   // first >= x
int j = upper_bound(all(a), x) - a.begin();   // first >  x
int countX = j - i;
int lessThanX = i;
int atMostX = j;
// largest element <= x: index j-1 (if j > 0)
// smallest element >= x: index i (if i < n)
```

Members on `set` / `map`: `s.lower_bound(x)` (O(log n)).

Example: CF 474B — Worms

Piles of a_i worms, labeled consecutively. For each query label q, which pile?

- Prefix sums: pref[i] = a_1 + … + a_i (Week 7)
- Answer = first i with pref[i] ≥ q → `lower_bound`

```
for each q:
    cout << lower_bound(pref.begin() + 1, pref.end(), q) - pref.begin() << '\n';
```

O((n + m) log n).

Binary search on the answer

Problem asks for the **smallest** x such that something is feasible.

1. Write `ok(x)`: "is x feasible?"
2. Show monotonicity: ok(x) ⇒ ok(x+1)
3. Find a lo with ok(lo) false and hi with ok(hi) true
4. Binary search for the boundary

Total: O(log(hi − lo) × cost of ok).

Factory Machines — the check

```
bool ok(long long T) {                  // can we make >= t products in time T?
    long long made = 0;
    for (long long k : machines) {
        made += T / k;
        if (made >= t) return true;   // early exit: avoids overflow!
    }
    return false;
}
```

Bounds: lo = 0 (makes 0 < t products), hi = min(k) · t ≤ 10^18 (fastest machine alone).

Factory Machines — the search

```
long long lo = 0, hi = *min_element(all(k)) * t;   // ok(lo)=F, ok(hi)=T
while (hi - lo > 1) {
    long long mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid; else lo = mid;
}
cout << hi << '\n';
```

O(n log(10^18)) ≈ 2·10^5 × 60 = 1.2·10^7 ✓

Minimize the maximum: CSES 1085 — Array Division

Split an array ($a_i \geq 1$) into k contiguous parts; minimize the largest part sum.

- ok(S): can we split into ≤ k parts each with sum ≤ S?
- Greedy check: extend the current part while it fits, else start a new one
- Monotone: a larger S never needs more parts
- lo = max(a) − 1 (infeasible), hi = Σ a (feasible)

Array Division — check

```
bool ok(long long S) {                        // assumes S >= max(a)
    int parts = 1;
    long long cur = 0;
    for (long long x : a) {
        if (cur + x > S) { parts++; cur = 0; }
        cur += x;
    }
    return parts <= k;
}
```

Pattern: **"minimize the max" / "maximize the min"** → binary search + greedy check.

Maximize the minimum (mirror image)

Example: place c cows in n stalls (positions), maximize the minimum distance.

- ok(d): can we place c cows with all gaps ≥ d? (greedy: leftmost first)
- Monotone **decreasing**: ok(d) ⇒ ok(d − 1)
- Want the **last true** → invariant: ok(lo) true, ok(hi) false, answer lo

```
d:   1 2 3 4 5 6
ok:  T T T T F F    -> answer 4
```

Binary search on real numbers

Stop by **iteration count**, not by `hi - lo > eps` (can loop forever with large values).

```
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; it++) {
    double mid = (lo + hi) / 2;
    if (ok(mid)) hi = mid; else lo = mid;
}
```

100 iterations shrink the range by 2^100 — far below double precision.

Example: cube root, "maximize average", geometry distances.

Integer square root: a classic trap

```
long long isqrt(long long n) {      // largest r with r*r <= n
    long long r = sqrtl(n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    return r;
}
```

Or binary search with ok(r) = r ≤ n / r (division avoids overflow of r·r).

`sqrt()` on doubles can be off by one for n near 10^18.

Complexity summary

Task	Cost
Search sorted array	O(log n)
Answer in [L, R], check O(n)	O(n log(R − L))
Real-valued, 100 iterations	O(100 · check)
q queries on sorted array	O((n + q) log n)

Check must be cheap; log(10^18) ≈ 60 — not 18!

Pitfalls

- Non-monotone predicate → binary search returns garbage
- `(lo + hi) / 2` overflow for big ints → `lo + (hi - lo) / 2`
- Bounds: is hi really feasible? is lo really infeasible?
- Overflow **inside** the check (sums of T / k_i) → early exit or cap
- Infinite loop with `lo = mid` and `mid = (lo + hi) / 2` rounding down → invariant template avoids it
- Calling `lower_bound` on an **unsorted** array

Practice problems

- CSES 1620 Factory Machines · 1085 Array Division · 1090 Ferris Wheel (warm-up)
- Codeforces 474B Worms · 706B Interesting drink · 1201C Maximum Median · 371C Hamburgers · 760B Frodo and pillows · 1613C Poisoned Dagger
- AtCoder ABC146 C Buy an Integer · ABC077 C Snuke Festival
- USACO 2016 Dec Silver — Counting Haybales · 2018 Dec Silver — Convention

Summary

- Binary search finds the boundary of a monotone predicate
- Invariant template: `lo` false, `hi` true, loop while `hi - lo > 1`
- STL: `lower_bound`, `upper_bound` for sorted data
- **Binary search on the answer**: ok(x) monotone + cheap check
- Next week: **two pointers & sliding windows** — another way to exploit monotonicity